BEAST 3 Overview

Modernising the Build, the Type System, and the Package Ecosystem

BEAST Core Developer Team

2026

Outline

1. Why Maven + JPMS?
2. Strongly Typed Spec System
3. New Distributions & Operators
4. Package System
5. Migration & Summary

The Big Picture

Three pillars of change in BEAST 3:

1. **Maven + JPMS**: modern build system and module isolation
2. **Strongly-typed spec system**: compile-time checked inputs and domain types
3. **New package distribution**: Maven Central JARs, module-based service discovery

Goal

Catch more errors at compile time, simplify dependency management, and make packages first-class citizens in the Java module system.

Problems We Wanted to Solve

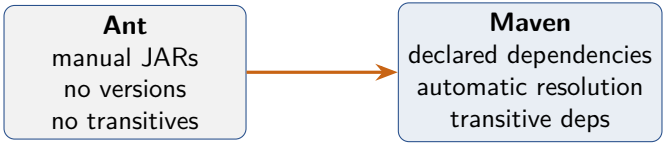
Build & dependencies

- Ant `build.xml`: manual JAR management, no transitive dependency resolution
- Adding a library means hand-copying JARs and editing classpaths
- No standard way to publish or consume packages

Classloading & isolation

- Custom classloader predates Java modules by 15+ years
- Source of "magic" that confuses new developers and breaks standard tooling
- Two packages bundling different versions of the same library can conflict

Why Maven? Dependency Management



- Declare what you need in `pom.xml`; Maven fetches, caches, and versions it
- Transitive dependencies resolved automatically: no more "which JARs do I need?"
- Reproducible builds: everyone gets the same dependency tree

Why Maven? Ecosystem & Tooling

- **IDE integration:** IntelliJ and Eclipse understand Maven projects natively: import, build, run, debug with zero configuration
- **CI/CD:** GitHub Actions, Jenkins, etc. all have first-class Maven support
- **Publishing:** packages can be published to Maven Central: the standard Java package registry, used by millions of projects
- **AI-assisted development:** Copilot, agentic coding tools understand Maven projects and JPMS modules; they don't understand bespoke build systems
- **Documentation & community:** vast ecosystem of tutorials, StackOverflow answers, and plugins

Moving to Maven means BEAST developers benefit from the entire Java ecosystem's tooling instead of maintaining our own.

Why JPMS Modules?

The **Java Platform Module System** (introduced in Java 9) gives us:

1. **Strong encapsulation**: each module declares what it exports; internal classes are truly hidden
2. **Reliable configuration**: missing dependencies are caught at startup, not at runtime via `ClassNotFoundException`
3. **Per-package isolation**: each BEAST package gets its own `ModuleLayer`; two packages can bundle different versions of the same library without conflict
4. **No split packages**: JPMS enforces that each Java package belongs to exactly one module, so good development practices are guaranteed rather than hoped for
5. **Service discovery**: `module-info.java` replaces classpath scanning and `version.xml` as the primary way to register BEASTObjects

JPMS replaces our custom classloader with a standard Java mechanism that works with, not against, the platform.

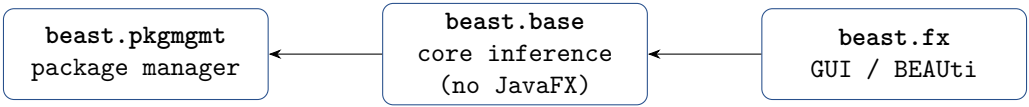
What Was Wrong with the Custom Classloader?

- Using BEAST as a **library** in other projects (e.g. LPhyBEAST) was nearly impossible: the custom classloader conflicts with the host application's class loading
- Debugging class-not-found errors required **BEAST-specific knowledge** that isn't transferable to other Java projects
- New developers and contributors faced a steep **onboarding barrier**

BEAST 3 fix: JPMS `ModuleLayer` provides per-package isolation out of the box. Standard Java, standard tooling, no magic.

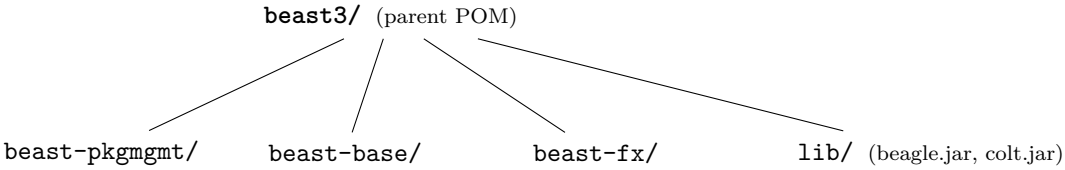
BEAST 3 Module Architecture

Three JPMS modules, each with its own `module-info.java`:



- `beast-base` has **no JavaFX dependency**: can run headlessly on clusters
- JavaFX is a **Maven dependency**, not bundled with the JDK
- All modules declared as `open module` (reflection OK for XML unmarshalling)

Project Structure



- Multi-module Maven project with a parent POM
- `mvn package` builds all three modules
- `beagle` and `colt` ship as modular JARs in `lib/`

The Type Safety Problem

In BEAST 2, almost everything is `Input<RealParameter>` or `Input<Function>`:

- Type errors only discovered at **runtime**: wrong parameter passed to wrong input
- Bounds (`lower="0"`, `upper="1"`) are stringly-typed XML attributes
- No way to express "this must be a positive real" in the Java API
- IDE cannot help you: everything looks like the same type

Goal

Make invalid models **unrepresentable** at compile time.

Strongly Typed Parameters & Domains

Typed parameters

- `RealScalarParam`: single real
- `RealVectorParam`: vector of reals
- `IntScalarParam` / `IntVectorParam`
- `BoolScalarParam` / `BoolVectorParam`
- `SimplexParam`: probability simplex

Domain types

- `PositiveReal`: replaces `lower="0"`
- `UnitInterval`: replaces `lower="0"` `upper="1"`
- `Real`: unbounded
- `NonNegativeInt`: replaces `lower="0"`
- `PositiveInt`: replaces `lower="1"`

- Domain is part of the **Java type**: compiler catches mismatches
- Legacy classes are **deprecated, not removed**: migrate incrementally

Before / After: Input Declaration

BEAST 2

```
public Input<RealParameter> kappaInput =
    new Input<>("kappa",
    "ts/tv ratio",
    Validate.REQUIRED);

// lower="0" only checked at runtime
// in the XML
```

BEAST 3

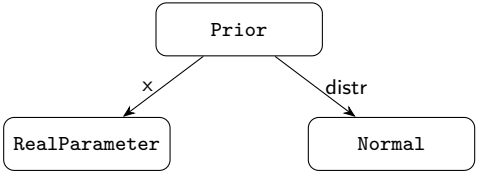
```
public Input<RealScalar<PositiveReal>>
    kappaInput = new Input<>("kappa",
    "ts/tv ratio",
    Validate.REQUIRED);

// PositiveReal is in the type!
// Compiler rejects wrong domain
```

"Distribution IS the Prior"

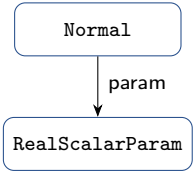
BEAST 2

`Prior` wraps distribution + parameter:



BEAST 3

Distribution **is** the prior:



Simpler model graph: no separate `Prior` wrapper needed.

New Distributions & Operators

New distributions

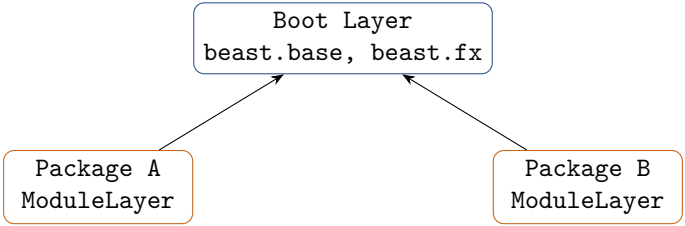
- Cauchy, GammaMean
- Bernoulli, IntUniform
- IID (scalar $\rightarrow$ vector),
- OffsetReal
- TruncatedReal

Operator changes

- New `IntervalScaleOperator`
- Dedicated `ScaleTreeOperator` (separated from `ScaleOperator`)
- Spec `UpDownOperator` replaces 3 BICEPS operators
- Simplified default tree operator set

Package System Overview

- **One ModuleLayer per package:** namespace isolation without losing visibility to the core API
- **Same code path for dev and deployment:** `module-info.java` `provides` declarations are the single source of truth
- **Graceful degradation:** modules with unsatisfied dependencies are silently skipped



Package Distribution: Maven Central & ZIP

Maven packages (recommended)

- Published to Maven Central
- Install via BEAUti or CLI
- Automatic dependency resolution
- `maven-packages.xml` tracks installs

ZIP packages (legacy, still works)

- Traditional CBAN archives
- One subdirectory per package
- `version.xml` for service discovery
- Self-describing directory structure

Both formats coexist. Loading precedence: boot layer → Maven → ZIP.

beast-package-skeleton

Ready-to-copy template project on GitHub:

- Maven `pom.xml` with BEAST 3 dependencies
- `module-info.java` with `provides` declarations
- Example distribution, operator, and tests
- BEAST XML using the strongly-typed spec API
- Maven Central release profile

Start here: `https://github.com/CompEvol/beast-package-skeleton`

Key Breaking Changes

Area	**BEAST 2**	**BEAST 3**
Build	Ant, Java 8/11	Maven, Java 25
Modules	Custom classloader	JPMS `ModuleLayer`
Parameters	`RealParameter`	Typed spec params + domains
Priors	`Prior` wrapper	Distribution is the prior
`StateNode`	implements `Function`	No longer a `Function`
`StateNode.scale()`	built-in	Removed; use `Scalable`
JavaFX	Bundled with JDK	Maven dependency

Legacy classes are **deprecated, not removed**: migrate incrementally.

Timeline

Now
Mar 2026
Stakeholders meeting (19 Mar)

Package migration
Apr–May
20+ packages across multiple groups

Release + paper
26 Jun 2026
Before SMBE Copenhagen

ISBA
26 Jun–5 Jul

Resources

Migration guide `scripts/migration-guide.md`: class mapping tables, step-by-step instructions

CHANGELOG `CHANGELOG.md`: full list of what changed and why

Package skeleton `https://github.com/CompEvol/beast-package-skeleton`

BEAST 3 repo `https://github.com/CompEvol/beast3`

Migration steps

1. Create `pom.xml` (see skeleton), target Java 25
2. Fix compile errors (`StateNode`/`Function`, `scale()`, `Evaluator`)
3. Add `module-info.java` with `provides` declarations
4. Migrate to spec types incrementally